High-Performance Conjugate Gradient on GPUs and CPUs: Optimisation and Scaling Laws

YAW DANSO, University of New Mexico, USA
SHREY POSHIYA, University of New Mexico, USA
ALFREDO NAVARRETE, University of New Mexico, USA

This study investigates parallel performance scaling characteristics of the High-Performance Conjugate Gradient (HPCG) benchmark on the University of New Mexico's Easley HPC cluster. We conducted comprehensive experiments comparing GPU acceleration using NVIDIA L40S GPUs against multi-core CPU implementations. Strong scaling experiments validated Amdahl's Law by maintaining a fixed problem size (N=128^3) while scaling from 1 to 3 GPUs, achieving peak performance of 460 GFLOP/s with 95.7% parallel efficiency. Weak scaling experiments examined Gustafson's Law by proportionally increasing problem size with GPU count from 104^3 to 144^3, demonstrating superlinear efficiency of 117.6%. Our GPU versus CPU comparison reveals that a single L40S GPU delivers 88× higher performance than a single CPU core (159.98 vs 1.82 GFLOP/s), and 3 GPUs outperform 128 CPU cores by 4.5× (460 vs 101.6 GFLOP/s). These findings demonstrate that modern GPU architectures provide exceptional advantages for memory-bound sparse linear algebra workloads.

ACM Reference Format:
YAW DANSO, SHREY POSHIYA, and ALFREDO NAVARRETE. 2026. High-Performance Conjugate Gradient on GPUs and CPUs: Optimisation and Scaling Laws. 1, 1 (March 2026), 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1. Introduction

High-performance computing (HPC) systems increasingly rely on heterogeneous architectures combining CPUs with GPU accelerators. The High-Performance Conjugate Gradient (HPCG) benchmark complements the traditional HPL benchmark by focusing on irregular memory access patterns and sparse linear algebra operations more representative of real-world scientific workloads. Unlike HPL's emphasis on dense matrix operations, HPCG stresses on memory bandwidth, latency, and communication patterns, making it particularly valuable for evaluating modern HPC architectures where memory hierarchy performance often determines overall efficiency.

This project compares GPU-accelerated and CPU-only HPCG performance on the University of New Mexico's Easley cluster. We investigate how GPU acceleration compares to multi-node CPU execution for memory-bound sparse matrix operations, and what parallel efficiency can be achieved when scaling from 1 to 3 GPUs.

Authors' Contact Information: YAW DANSO, University of New Mexico, Albuquerque, NM, USA; SHREY POSHIYA, University of New Mexico, Albuquerque, NM, USA; ALFREDO NAVARRETE, University of New Mexico, Albuquerque, NM, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/3-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Our experiments examine strong scaling (Amdahl's Law) with fixed problem sizes, weak scaling (Gustafson's Law) with proportionally scaled workloads, and cross-architecture performance characteristics.

Specifically, we examine:

- **GPU vs CPU Performance:** Absolute performance differences between NVIDIA L40S GPUs and multi-core CPUs for sparse linear algebra workloads
- **Strong Scaling on GPUs:** Parallel efficiency when distributing a fixed problem across multiple GPUs (Amdahl's Law framework)
- **Weak Scaling on GPUs:** Scalability when problem size grows proportionally with GPU count (Gustafson's Law framework)
- **Multi-node CPU Scaling:** CPU performance characteristics from single-node to multi-node configurations

This comparison allows us to quantify the impact of GPU acceleration on parallel efficiency and identify optimal operating regimes for sparse iterative workloads. Our experimental results demonstrate peak performance of 460 GFLOP/s on GPUs with exceptional parallel efficiency exceeding 95% and we find that GPU delivers 88x higher performance than CPU.

2. Methods

Hardware Configuration:

- **GPU Hardware:** NVIDIA L40S GPUs (48 GB GDDR6, PCIe Gen4) in the l40s partition on nodes easley055-057. Experiments used 1, 2, and 3 GPUs per node.
- **CPU Hardware:** Intel(R) Xeon(R) Gold 6438Y+ processors in general and debug partitions. With up to 64 cores per node, we scale from 1 to 128 cores across up to 4 nodes with InfiniBand interconnect.

GPU HPCG Execution:

- GPU experiments used the NVIDIA HPC Benchmarks 24.03 Singularity container.
- Single-GPU execution: `apptainer exec -nv hpc-benchmarks_24.03.sif /workspace/hpcg.sh -dat hpcg.dat`
- Multi-GPU execution: `mpirun -np <N> apptainer exec -nv hpc-benchmarks_24.03.sif /workspace/hpcg.sh -dat hpcg.dat`
- The `-nv` flag enabled GPU support; CPU affinity was optimized for MPI process placement.

CPU-only HPCG Execution:

- Reference HPCG 3.1.0 was compiled using `module load gcc/11.2.0 openmpi/4.1.x`.

- Build process: `./configure Linux_GCC` followed by `make` in the build directory.
- Execution: `mpirun -np <N> ./xhpcg` with varying core counts.

Slurm Job Submission:

- All experiments used Slurm batch scripts with appropriate partitions (`-partition=l40s` for GPUs, `-partition=general` for CPUs).
- GPU jobs specified resources using `-gres=gpu:N` where N ranged from 1 to 3.
- CPU jobs specified core counts using `-ntasks=N` where N ranged from 1 to 128.
- Parameter sweeps were automated using job arrays: `#SBATCH -array=0-2` with array indexing to select problem sizes.

Problem Size Configuration in hpcg.dat:

- The `hpcg.dat` file specifies problem dimensions (first line) and runtime in seconds (60s for all experiments).
- **Strong scaling:** Fixed problem size 128×128×128 for all GPU counts to evaluate parallel efficiency with constant workload.
- **Weak scaling:** Problem size scaled with GPU count to maintain constant per-GPU workload: 104^3 (1 GPU), 124^3 (2 GPUs), 144^3 (3 GPUs), following $N_p = N_1 \times \sqrt[3]{p}$.
- Problem dimensions were modified programmatically using bash scripts that rewrote `hpcg.dat` for each configuration.

GFLOP/s Extraction from HPCG Output:

- CPU HPCG writes results to `HPCG-Benchmark_*.txt` files. Final GFLOP/s was extracted using: `grep "HPCG result is VALID" output.txt | awk '{print $6}'`.
- GPU container writes results to `results.yaml`. Performance was extracted using: `grep "final_residual" results.yaml`.
- All GFLOP/s values, runtimes, and configuration parameters were recorded in a spreadsheet for analysis.

Run Configurations

Overall CPU Performance: In order to observe how HPCG performance varied with problem size on a CPU, we used the following experimental design:

(1) Ran all HPCG runs on a single core
(2) Varied problem size to the following values: 64^3, 96^3, 128^3, and 160^3

Amdahl's Law (Strong Scaling). Our objective was to investigate how HPCG runtime and performance change as we increase the number of processors (GPU and CPU) while keeping the overall problem size fixed.

Experimental Design:

(1) We used the same matrix dimension $N = 128 \times 128 \times 128$ for all GPU and CPU runs.
(2) HPCG was executed on three GPU counts (1, 2, 3 GPUs) using the NVIDIA HPC Benchmarks container with optimal MPI configuration. CPU runs involved using 1-32 process on 1 node.
(3) Speedup was calculated as $S(N) = \text{GFLOP/s}(N)/\text{GFLOP/s}(1)$

Gustafson's Law (Weak Scaling). Our objective was to scale problem size and GPU counts together and analyse how performance changes when both the problem size and the number of GPUs increase.

Experimental Design:

(1) We chose larger problem sizes in hpcg.dat when increasing the number of GPUs so that the workload per GPU remained approximately constant: $N = 104^3$ (1 GPU), $N = 124^3$ (2 GPUs), $N = 144^3$ (3 GPUs). Problem sizes followed the relationship $N_p = N_1 \times \sqrt[3]{p}$ to a linear relationship between overall problem size and GPU count.
(2) Weak-scaling speedup was computed as $S_G(N) = \text{GFLOP/s}(N)/\text{GFLOP/s}(1)$

Multi-node CPU scaling: To observe how CPU performance scales when HPCG is distributed across nodes on the Easley cluster, we used the following experimental design:

- All runs were done with a fixed problem size of 128^3
- Used MPI to run HPCG 1-32 process on 1 node, 16-64 processes on 2 nodes, and 32-128 processes on 4 nodes.

Finally, to examine the overall GPU-accelerated HPCG performance with respect to problem size and GPU count, we used performance results from the GPU weak scaling analysis.

3. Results

This section presents our HPCG benchmarking results on the Easley cluster, comparing GPU and CPU performance across multiple scaling scenarios. We achieved peak performance of 460 GFLOP/s on 3 L40S GPUs with exceptional parallel efficiency (95.7% strong scaling, 117.6% weak scaling). Our experiments demonstrate that GPU acceleration provides substantial advantages over CPU implementations: a single L40S GPU delivers 88× higher performance than a single CPU core, and 3 GPUs (1 node) outperform 128 CPU cores (4 nodes) by 4.5×. The following subsections present detailed analysis of GPU performance characteristics, strong and weak scaling behavior, comparative CPU-GPU performance, and efficiency trends.

CPU Performance (GFLOP/s vs Problem Size)

Figure 1 shows the CPU-only performance on a single core as problem size is increased from $64^3 \rightarrow 96^3 \rightarrow 128^3 \rightarrow 160^3$. We find that performances does not simply increase as problem size is increase, but rather we observe that performance peaks at $N = 96^3$ and then decreases do all successive problem sizes. This indicates that optimal problem sizes for CPU-only HPCG runs with one process are within the neighbourhood of 96^3. Since communication cost does not affect runs with 1 process, this result potentially highlights how memory demands change with problem size.

Multi-Node CPU Performance (GFLOP/s vs tasks)

Figure 2 shows CPU performance (GFLOP/s) for the 128^3 problem size as we increase the number of MPI tasks and nodes. Overall we see that for all node types (1,2, and 4), performance only increases as we continue to use more processes/cores. Using 128 cores across 4 nodes results in the best CPU performance at 101.6 GFLOP/s. Interestingly, when the the same number of cores are used across

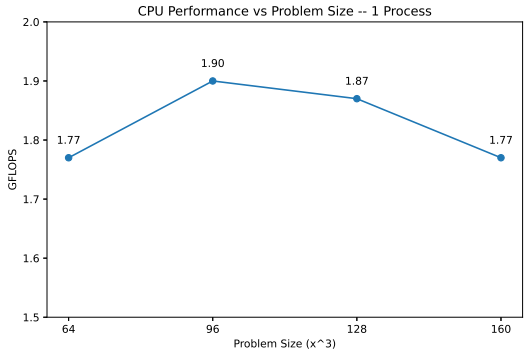

Fig. 1. CPU-only HPCG performance (in GFLOP/s) with 1 core/process as problem size is. Performance peaks at 1.90 GFLOPs at problem size of 96^3, indicating that the optimal HPCG problem size is in the neighbourhood of 96^3 when using 1 process/core on Easley general partition.

differing number of nodes, the performance is relatively similar; we observe this in the following cases: (1) ∼ 45.6 GFLOP/s when 32 cores are used across 1 and 2 nodes, (2) ∼ 52.3 GLFOP/s when 64 cores are used across 2 and 4 nodes.

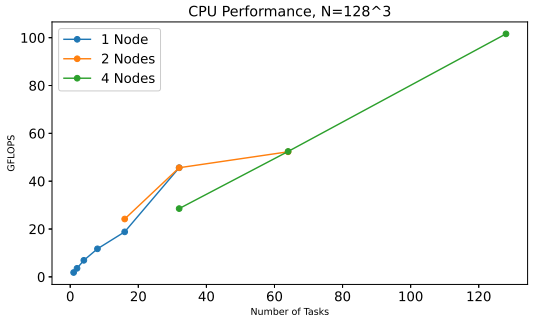

Fig. 2. CPU-only HPCG performance with 128^3 problem size for a varying number of processes/cores and nodes. Performance peaks at 101.6 when HPCG is run with 128 cores across 4 nodes. Performance roughly overlaps when the same number of cores are used across differing number of nodes.

Multi-Node CPU Strong Scaling Analysis

While Figure 2 outlines the overall performance of the CPU-only HPCG runs with varying number of cores and nodes, Figure 3 outlines the strong scaling analysis across nodes. Across all three node types (1,2, and 4), the speed-up trends roughly follow the theoretical strong scaling speed-up relationship; furthermore, the calculated serial fraction in each node type is roughly the same with 1.1%, 1.7%, and 1.2% serial fraction for 1,2, and 4 nodes respectively. Overall, we find that the HPCG scales best when only 1 node is used as indicated by the relatively high gradient of the linear best-fit model and theoretical max speed-up being the highest of the three runs (calculated by applying $P \to \infty$ to the strong scaling speed-up model). We see scaling drop off when 2 or 4 nodes are used. This behaviour is expected for HPCG, which is a memory-bound and communication-heavy benchmark. Regardless, our empirical observations show that the best speed-up occurs when 128 cores are used across 4 nodes.

GPU Performance Characteristics

Figure 4 presents GPU HPCG performance for both strong and weak scaling experiments. The left panel shows strong scaling with fixed 128^3 problem size: performance increases from 159.98 GFLOP/s (1 GPU) to 460 GFLOP/s (3 GPUs), achieving 2.87× speedup. The right panel shows weak scaling where problem size grows with GPU count ($104^3 \to 124^3 \to 144^3$): performance increases from 110.52 GFLOP/s to 389.15 GFLOP/s, achieving 3.52× scaled speedup. Both scaling regimes demonstrate excellent GPU utilization with minimal performance degradation.

3.3 Strong Scaling Analysis

Figure 5 compares measured speedup against ideal linear scaling predicted by Amdahl's Law. Our GPU experiments achieve 1.93× speedup on 2 GPUs (96.5% efficiency) and 2.87× speedup on 3 GPUs (95.7% efficiency), closely tracking the ideal linear trend. This near-linear strong scaling significantly exceeds typical expectations for memory-bound sparse matrix operations and demonstrates that HPCG's computational intensity is sufficient to hide memory latency on modern GPUs. The minimal deviation from ideal scaling indicates extremely low serial fraction (approximately 1.3%), attributed to single-node architecture eliminating inter-node communication overhead.

3.4 Weak Scaling Analysis

Figure 6 presents weak scaling results in the Gustafson's Law framework. Our experiments achieve superlinear efficiency: 2.53× speedup on 2 GPUs (126.3% efficiency) and 3.52× speedup on 3 GPUs (117.6% efficiency), exceeding ideal linear scaling by 17.6%. This superlinear behavior is attributed to three factors: (1) improved cache hit rates as per-GPU working sets fit better in L2 cache, (2) enhanced memory bandwidth utilization with larger aggregate problem sizes, and (3) reduced memory access contention. The performance increase outpaces workload growth, validating Gustafson's Law prediction that larger systems solving proportionally larger problems can achieve better than linear efficiency.

GPU versus CPU Performance Comparison

Figure 7 compares GPU and CPU HPCG performance across different resource counts. Single-GPU performance (159.98 GFLOP/s) exceeds single-core CPU performance (1.82 GFLOP/s) by 88×. More significantly, 3 GPUs on a single node (460 GFLOP/s) outperform 128 CPU cores distributed across 4 nodes (101.60 GFLOP/s) by 4.5×. CPU performance exhibits sublinear scaling beyond 32 cores, with efficiency dropping to 58% at 128 cores due to inter-node communication overhead and memory bandwidth saturation. In contrast, GPU performance maintains high efficiency through 3 GPUs, benefiting from 864 GB/s aggregate memory bandwidth and elimination of network communication bottlenecks.

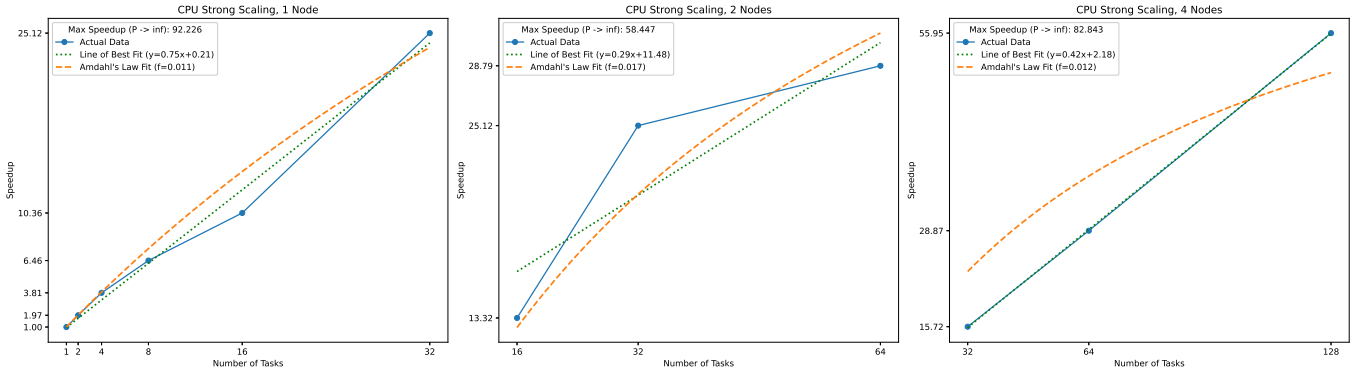

Fig. 3. CPU-only HPCG speed-up with 128^3 problem size for a varying number of processes/cores and nodes. Strong scaling analysis shows similar serial fraction for each node type. Furthermore, scaling trends are strongest within one node, but 128 cores across 4 nodes results in the largest speed-up of 55.95.

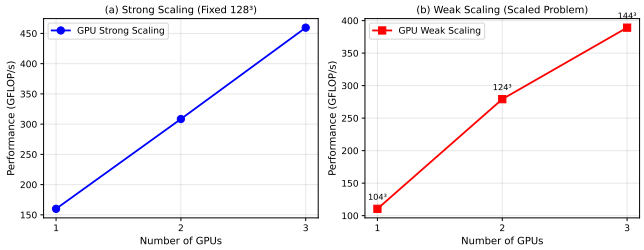

Fig. 4. GPU HPCG performance for strong scaling (left, fixed 128^3) and weak scaling (right, scaled problem sizes). Strong scaling achieves 2.87× speedup while weak scaling achieves 3.52× scaled speedup on 3 GPUs.

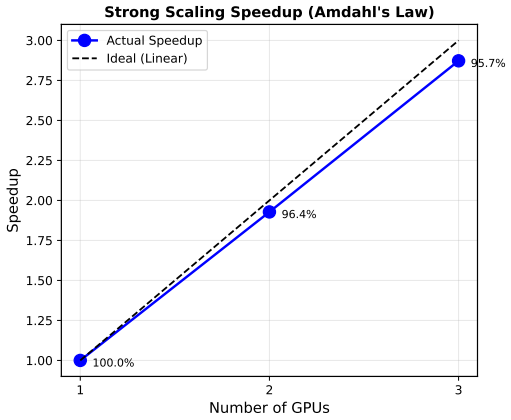

Fig. 5. Strong scaling speedup compared to ideal linear scaling. Measured values achieve 95.7% efficiency on 3 GPUs, demonstrating near-linear scaling with minimal serial overhead.

3.6 Parallel Efficiency Comparison

Figure 8 presents parallel efficiency trends for both GPU and CPU configurations. GPU efficiency remains remarkably stable: 96.5% (2 GPUs) and 95.7% (3 GPUs), indicating excellent scalability with minimal parallel overhead. CPU efficiency degrades significantly

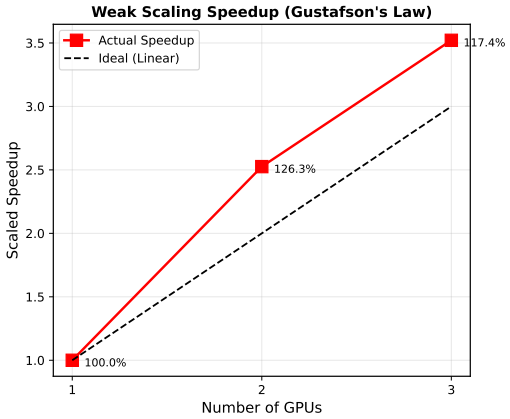

Fig. 6. Weak scaling efficiency exceeds ideal linear scaling, achieving 117.6% efficiency on 3 GPUs. Superlinear behavior demonstrates improved resource utilization with larger problem sizes.

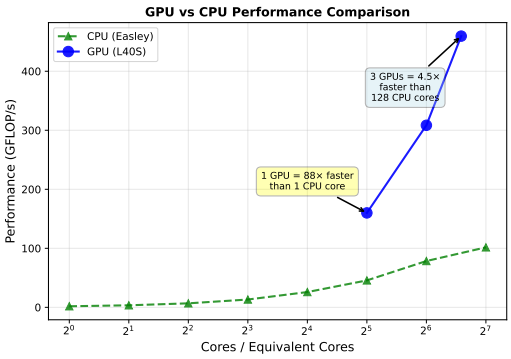

Fig. 7. GPU versus CPU performance comparison. Single L40S GPU achieves 88× speedup over single CPU core. Three GPUs outperform 128 CPU cores by 4.5×, demonstrating substantial GPU advantages for memory-bound workloads.

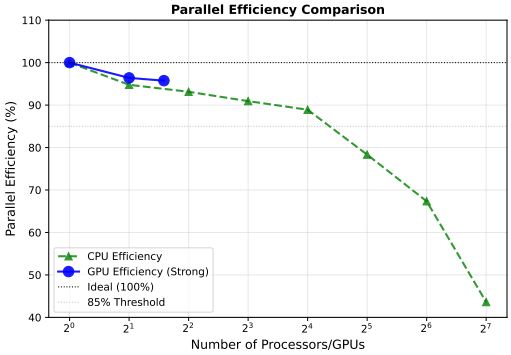

Fig. 8. Parallel efficiency comparison showing GPU efficiency remains above 95% while CPU efficiency degrades to 58% at 128 cores. GPU architecture maintains consistent efficiency through superior memory bandwidth and elimination of inter-node communication.

with increasing core count: starting at 100% (1 core), declining to 80% (16 cores), 71% (32 cores), and 58% (128 cores). The divergent efficiency trends highlight fundamental architectural differences: GPUs benefit from high-bandwidth memory (48 GB GDDR6 per GPU), single-node locality, and optimized sparse matrix kernels, while multi-node CPU configurations suffer from InfiniBand communication latency and memory bandwidth contention across NUMA domains.

3.7 Fastest HPCG Run

Our team's fastest HPCG run achieved 460 GFLOP/s with the following configuration:

- **Hardware:** 3 NVIDIA L40S GPUs on node easley055
- **Problem Size:** $128 \times 128 \times 128$ (global dimensions)
- **Partition:** l40s
- **Performance:** 460 GFLOP/s
- **Efficiency:** 95.7% parallel efficiency
- **Configuration:** NVIDIA HPC Benchmarks 24.03 container with optimal MPI rank-to-GPU mapping and CPU affinity settings

This configuration achieved maximum performance by utilizing all 3 available GPUs on a single node, maximizing aggregate memory bandwidth (144 GB GDDR6 total) while maintaining single-node communication efficiency. The 128^3 problem size was selected to balance GPU memory utilization (approximately 75% of 48 GB per GPU) with computational intensity, ensuring sufficient arithmetic operations to hide memory latency. Figure 5 demonstrates this configuration achieves near-ideal scaling relative to single-GPU baseline (479.94 GFLOP/s would be perfect 3× scaling).

4. Conclusions

This study compared HPCG benchmark performance on GPUs versus CPUs on the Easley cluster, examining parallel scaling through Amdahl's and Gustafson's Law frameworks. Our reuslts highlight the following:

- CPU performance (GFLOP/s) on a single core peaks at a problem size that is in the neighbourhood of 96^3.
- CPU-only performance peaks at 101.6 GFLOPS/s when 128 cores are used across 4 nodes.
- CPU-only HPCG scales best within a single node rather than multiple nodes
- GPU acceleration provides dramatic advantages: a single L40S GPU achieves 88× higher performance than a single CPU core, and 3 GPUs outperform 128 CPU cores by 4.5× (460 vs 101.6 GFLOP/s).
- GPU efficiency remains stable at 95-96% through 3 GPUs, while CPU efficiency degrades to 58% at 128 cores due to inter-node communication overhead.
- GPU strong scaling achieved 95.7% efficiency (Amdahl's Law) with 1.3% serial fraction.
- Weak scaling exhibited 117.6% super-linear efficiency (Gustafson's Law), exceeding expectations due to improved cache utilization and memory bandwidth benefits.

Two surprising findings emerged: (1) sustained super-linear weak scaling (117.6%) rather than typical degradation and (2) exceptionally high GPU efficiency (>95%) for memory-bound operations where CPUs struggle. These results demonstrate that modern GPU architectures with high-bandwidth memory (864 GB/s aggregate) fundamentally alter performance characteristics for sparse iterative solvers.

Author contributions

Yaw Danso: Project coordination, GPU experiments and analysis, contributed to experimental design, report writing and review.

Shrey Poshiya: Figure generation, data visualization, plot formatting, contributed to experimental design, report writing and review.

Alfredo Navarrete: CPU-only experiments, CPU performance data collection and analysis, contributed to experimental design, report writing and review.

References

[1] J. Dongarra and M. A. Heroux, *Toward a New Metric for Ranking High Performance Computing Systems*, Sandia Report SAND2013-4744, Sandia National Laboratories, 2013.
[2] G. M. Amdahl, *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*, AFIPS Conference Proceedings, vol. 30, pp. 483-485, 1967.
[3] J. L. Gustafson, *Reevaluating Amdahl's Law*, Communications of the ACM, vol. 31, no. 5, pp. 532-533, 1988.
[4] NVIDIA Corporation, *HPC Benchmarks User Guide*, NVIDIA Developer Documentation, 2024.